Exercícios — Ponteiros

*Os exercícios marcados com **[Formato de prova]** seguem o modelo das avaliações: resolva-os primeiro no papel, sem o computador, e só depois confira executando o código. Nos demais, execute e teste o seu programa com diferentes entradas.*

Nível Fácil

1) Escreva um programa que declare uma variável `int`, uma `double` e uma `char`. Antes de executar, anote quantos bytes você espera para cada uma. Depois imprima, para cada variável, o seu tamanho (`sizeof`) e o seu endereço (`&`). Execute o programa duas vezes: o que você observa nos endereços de uma execução para a outra?

2) Dado o trecho abaixo:

```
int v = 5;
int *p = &v;
```

   a) Escreva as instruções que imprimem: o valor de `v`, o endereço de `v`, o valor de `p`, o endereço de `p` e o valor apontado por `p`.
   b) Usando **apenas** o ponteiro `p`, altere o valor de `v` para 20 e imprima `v`.
   c) Entre as cinco saídas do item (a), quais duas são iguais? Explique em um comentário no código.

3) Sem executar, preveja as três linhas impressas pelo programa abaixo. Desenhe as caixas `v`, `x`, `p1` e `p2` (com setas) a cada instrução. Depois execute e confira.

```
int *p1, *p2, v = 5, x = 3;
p1 = &v;
p2 = &x;
*p1 = *p2;
cout << *p1 << " " << *p2 << "\n";
*p1 = 9;
cout << *p1 << " " << *p2 << "\n";
p2 = p1;
*p2 = 10;
cout << *p1 << " " << *p2 << " " << v << " " << x;
```

4) Escreva a função `void troca(int *a, int *b)`, que troca os valores das duas variáveis cujos endereços recebe. No `main`, leia dois inteiros, chame a função e imprima os valores antes e depois da troca. Em um comentário, explique por que `void troca(int a, int b)` **não** funcionaria.

5) **[Formato de prova]** Qual é a saída impressa na tela pelo código abaixo?

```
int a = 10, b = 20;
int *p = &a;
int *q = &b;
*p = *q + 5;
q = p;
*q = *q + 1;
cout << a << " " << b << " " << *p << " " << *q;
```

A) 25 20 25 25
B) 26 20 26 26
C) 26 21 26 26
D) 10 20 10 10
E) Erro de compilação

Nível Médio

6) Um ponteiro pode dizer a uma função **onde depositar** o seu resultado. Escreva a função `void hm(int mnts, int *h, int *m)`, que converte `mnts` minutos em horas e minutos, de modo que `m` seja menor que 60 e `60*h + m` seja igual a `mnts`. No `main`, leia os minutos, chame a função e imprima o resultado no formato `2h15min`.

7) Escreva a função `void mm(int v[], int n, int *min, int *max)`, que recebe um vetor com `n` elementos e deposita nas variáveis apontadas por `min` e `max` o menor e o maior elemento do vetor. No `main`, leia um vetor de 10 inteiros, chame a função e imprima o menor e o maior valor.

8) Escreva as duas funções abaixo **sem usar o operador []** (use apenas `*` e aritmética de ponteiros). No `main`, leia um vetor de 8 inteiros, chame as duas funções e imprima a soma e o vetor resultante.

   a) `int soma(int *v, int n)`: devolve a soma dos elementos, acessando-os com `*(v + i)`.

   b) `void zeraNegativos(int *v, int n)`: substitui por zero os elementos negativos, usando um ponteiro `c` que percorre o vetor de `v` até `v + n`.

9) Suponha que `v` é um vetor de `int`, que cada `int` ocupa 4 bytes e que o endereço de `v[0]` é 55000. Responda:

   a) Qual é o valor da expressão `v + 3`?

   b) E se `v` fosse um vetor de `double` (8 bytes)? Qual seria o valor de `v + 3`?

   c) Se `i` é uma variável inteira, descreva em português a sequência de operações que deve ser executada para calcular o valor da expressão `&v[i + 9]`.

   d) Qual é a diferença conceitual entre as expressões `v[3]` e `v + 3`?

10) Considere a struct abaixo. Escreva a função `void reajusta(Aluno *a, float pontos)`, que soma `pontos` à nota do aluno (limitando a nota em 10) e atualiza o campo `aprovado` (verdadeiro se a nota for maior ou igual a 6). No `main`, crie um vetor com 3 alunos, reajuste todos em 1 ponto e imprima nome, nota e situação (aprovado ou reprovado) de cada um.

```
struct Aluno {
    string nome;
    float nota;
    bool aprovado;
};
```

11) Escreva uma função **template** `troca` que receba dois **ponteiros** para o mesmo tipo `T` e troque os valores apontados. Use-a, no `main`, para trocar dois `int`, dois `double` e duas `string`. Depois, use-a para trocar dois `Aluno` (struct da questão anterior).

12) **[Formato de prova]** Qual é a saída impressa na tela pelo código abaixo?

```
int v[5] = {10, 20, 30, 40, 50};
int *p = v + 1;
*p = *p + 5;
p = p + 2;
*p = *(p - 1) + *(p + 1);
cout << v[1] << " " << v[3] << " " << v[4] << " " << *p;
```

A) 25 80 50 80
B) 25 70 50 70
C) 20 80 50 80
D) 25 80 50 50
E) 25 40 50 40

Nível Difícil

13) Uma função também pode **devolver um ponteiro**. Escreva `int* busca(int *v, int n, int alvo)`, que devolve o endereço da **primeira** ocorrência de `alvo` no vetor ou `nullptr` se ele não existir. No `main`, leia um vetor de 10 inteiros e um valor a buscar. Se for encontrado, imprima a **posição** dele no vetor (dica: subtraia `p - v`) e some 100 ao elemento **usando o ponteiro devolvido**. Se não for encontrado, informe ao usuário.

14) Escreva duas funções que invertem a ordem dos elementos de um vetor **sem usar []**, recebendo o endereço do primeiro elemento (`ini`) e o do último (`fim`): `void inverte(int *ini, int *fim)`, com um laço, e `void inverteRec(int *ini, int *fim)`, **recursiva**. Exemplo: o vetor 1 2 3 4 5 vira 5 4 3 2 1. No `main`, teste as duas versões.

15) O arquivo `turma.txt` contém, em cada linha, o nome (sem espaços) e a nota de um aluno, por exemplo `Ana 7.5`. Há no máximo 50 linhas. Usando a struct `Aluno` (nome e nota), escreva um programa que: (i) leia o arquivo para um vetor de `Aluno`, informando se ele não puder ser aberto; (ii) use uma função `Aluno* melhor(Aluno *turma, int n)`, que devolve o **endereço** do aluno com maior nota; (iii) aumente em 0,5 a nota desse aluno **através do ponteiro devolvido**; (iv) grave a turma atualizada no arquivo `turma_atualizada.txt`, no mesmo formato.

```
turma.txt:
Ana 7.5
Bruno 9
Carla 6
```

16) **[Formato de prova]** Qual é a saída impressa na tela pelo código abaixo?

```
void misterio(int *a, int *b) {
    int t = *a;
    *a = *b + t;
    *b = t;
}

int main() {
    int x = 2, y = 5;
    misterio(&x, &y);
    misterio(&y, &x);
    cout << x << " " << y;
    return 0;
}
```

A) 2 9
B) 9 2
C) 7 2
D) 7 9
E) 2 7

17) **[Formato de prova]** (1,0 ponto) Escreva uma função **template** `maior`, que receba um vetor de elementos de tipo genérico `T` e a quantidade `n` de elementos e **devolva um ponteiro** para o maior elemento (o tipo de retorno é `T*`). Escreva apenas a função, sem o `main`. Em seguida, diga o que o `main` abaixo imprime.

```
int main() {
    int v[3] = {4, 9, 2};
    string s[3] = {"pera", "uva", "maca"};
    *maior(v, 3) = 0;
    cout << v[0] << v[1] << v[2] << " " << *maior(s, 3);
    return 0;
}
```